

Name : Sneha Subhashree

Regd no. : 2141019389

LSP - Capstone3 - System Monitor Tool

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

Day 2: Display process list with CPU and memory usage.

Day 3: Implement process sorting by CPU and memory usage.

Day 4: Add functionality to kill processes.

Day 5: Implement real-time update feature to refresh data every few seconds.

Day 1: Design UI layout and gather system data using system calls.

Code :

```
#include <iostream>
using namespace std;
#include <sys/sysinfo.h>
void displayMemoryInfo(){
    struct sysinfo info;
    if (sysinfo(&info)==0){
        cout<< "Total used RAM (info.totalram in MB):
"<<info.totalram/(1024*1024)<<"MB\n";
        cout<< "Total used RAM (info.totalram in GB):
"<<(info.totalram/(1024*1024))/1024<<"GB\n";
        cout<< "Total unused RAM (info.freeram in MB):
"<<info.freeram/(1024*1024)<<"MB\n";
        cout<< "Total shared RAM (info.sharedram in MB):
"<<info.sharedram/(1024*1024)<<"MB\n";
    }
}
int main(){
    displayMemoryInfo();
    return 0;
}
```

Output :

```
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ task1.cpp -o t1
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t1
Total used RAM (info.totalram in MB): 7681MB
Total used RAM (info.totalram in GB): 7GB
Total unused RAM (info.freeram in MB): 4531MB
Total shared RAM (info.sharedram in MB): 261MB
```

Day 2: Display process list with CPU and memory usage.

Code :

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <thread>
#include <chrono>
using namespace std;
struct CPUData{
    long user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice;
};
CPUData getCPUData(){
    ifstream file("/proc/stat");
    string line;
    CPUData cpu={};
    if (file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string cpuLabel;
        ss >> cpuLabel >> cpu.user >> cpu.nice >> cpu.system >> cpu.idle >>
cpu.iowait >> cpu.irq >> cpu.softirq >> cpu.steal >> cpu.guest >> cpu.guest_nice;
    }
    return cpu;
}
double calculateCPUUsage (CPUData prev, CPUData current){
    long prevIdle= prev.idle + prev.iowait;
    long currIdle= current.idle + current.iowait;
    long prevTotal = prev.user + prev.nice + prev.system + prev.idle + prev.iowait +
prev.irq + prev.softirq + prev.steal;
    long currTotal = current.user + current.nice + current.system + current.idle +
current.iowait + current.irq + current.softirq + current.steal;
```

```

        long totalDiff = currTotal - prevTotal;
        long idleDiff = currIdle - prevIdle;
        return ((totalDiff - idleDiff) * 100.0) / totalDiff;
    }
    int main(){
        CPUData cpu= getCPUData();
        cout << "User: " << cpu.user << endl;
        cout << "Nice: " << cpu.nice << endl;
        cout << "System: " << cpu.system << endl;
        cout << "Idle: " << cpu.idle << endl;
        cout << "IOWait: " << cpu.iowait << endl;
        cout << "IRQ: " << cpu irq << endl;
        cout << "SoftIRQ: " << cpu.softirq << endl;
        cout << "Steal: " << cpu.steal << endl;
        cout << "Guest: " << cpu.guest << endl;
        cout << "Guest Nice: " << cpu.guest_nice << endl;
        CPUData prevData = getCPUData();
        this_thread::sleep_for(chrono::seconds(1));
        CPUData currData = getCPUData();
        double cpuUsage= calculateCPUUsage (prevData, currData);
        cout << "CPU Usage: "<<cpuUsage<<endl;
        return 0;
    }
}

```

Output :

```

student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ --std=c++17 task2.cpp -o t2
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t2
User: 18341
Nice: 13
System: 2862
Idle: 326511
IOWait: 3420
IRQ: 0
SoftIRQ: 32
Steal: 0
Guest: 0
Guest Nice: 0
CPU Usage: 12.427

```

Code :

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
using namespace std;
namespace fs = std::filesystem;
bool isNumber(const string &s){
    return !s.empty() && all_of(s.begin(), s.end(), ::isdigit);
}

string getProcessName(int pid){
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line, processName;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;
        while(ss >> token){
            count++;
            if(count==2){
                processName=token;
                break;
            }
        }
    }
    return processName;
}

int main(){
    cout<<"Active processes: "<<endl;
    for (const auto &entry : fs::directory_iterator("/proc")){
        if(entry.is_directory()){
            string filename=entry.path().filename().string();
            if (isNumber(filename)){
                int pid=stoi(filename);
                string processName = getProcessName(pid);
                cout<<"PID: "<<pid<<"|Name: " << processName<<endl;
            }
        }
    }
}
```

```

    }
}
return 0;
}

```

Output :

```

student@D001-38: ~/Downloads/2141019389_LSP_Capstone3
student@D001-38:~$ cd Downloads
student@D001-38:~/Downloads$ cd 2141019389_LSP_Capstone3
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ --std=c++17 task2_2.cpp -o t02
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t02
Active processes:
PID: 1|Name: (systemd)
PID: 2|Name: (kthreadd)
PID: 3|Name: (rcu_gp)
PID: 4|Name: (rcu_par_gp)
PID: 5|Name: (slub_flushwq)
PID: 6|Name: (netns)
PID: 8|Name: (kworker/0:0H-events_highpri)
PID: 10|Name: (mm_percpu_wq)
PID: 11|Name: (rcu_tasks_rude_)
PID: 12|Name: (rcu_tasks_trace)
PID: 13|Name: (ksoftirqd/0)
PID: 14|Name: (rcu_sched)
PID: 15|Name: (migration/0)
PID: 16|Name: (idle_inject/0)
PID: 18|Name: (cpuhp/0)
PID: 19|Name: (cpuhp/1)
PID: 20|Name: (idle_inject/1)
PID: 21|Name: (migration/1)
PID: 22|Name: (ksoftirqd/1)
PID: 24|Name: (kworker/1:0H-events_highpri)
PID: 25|Name: (cpuhp/2)
PID: 26|Name: (idle_inject/2)
PID: 27|Name: (migration/2)
PID: 28|Name: (ksoftirqd/2)
PID: 30|Name: (kworker/2:0H-events_highpri)
PID: 31|Name: (cpuhp/3)
PID: 32|Name: (idle_inject/3)
PID: 33|Name: (migration/3)
PID: 34|Name: (ksoftirqd/3)
PID: 36|Name: (kworker/3:0H-events_highpri)
PID: 37|Name: (cpuhp/4)
PID: 38|Name: (idle_inject/4)
PID: 39|Name: (migration/4)
PID: 40|Name: (ksoftirqd/4)
PID: 42|Name: (kworker/4:0H-events_highpri)
PID: 43|Name: (cpuhp/5)
PID: 44|Name: (idle_inject/5)
PID: 45|Name: (migration/5)
PID: 46|Name: (ksoftirqd/5)

```

Day 3: Implement process sorting by CPU and memory usage.

Code :

```

#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>

```

```

#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
    int pid;
    string name;
    double cpuUsage;
    long memoryUsage;
};
string readFileValue(const string &path){
    ifstream file(path);
    string value;
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}
double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}
ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;

        while(ss >> token){

```

```

        count++;
        if(count==2) pinfo.name=token;
        else if(count==14) utime=stol(token);
        else if(count==15) stime=stol(token);
        else if(count==22) starttime=stol(token);
    }
}

ifstream memFile("/proc/" + to_string(pid) + "/status");
if (memFile.is_open()){
    string key, value,unit;
    while(memFile >> key >> value >> unit){
        if (key=="VmRSS"){
            pinfo.memoryUsage=stol(value);
            break;
        }
    }
}

long total_time = utime + stime;
double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
return pinfo;

}

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);

processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
}

```

```

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){
    vector <ProcessInfo> processes = getAllProcesses();
    sortProcesses(processes,true);
    cout<<"PID\tCPU%\tMemory (kB)\tName\n";
    for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryU
sage<<"%\t"<<processes[i].name<<"%\n";
    }
    return 0;
}

```

Output :

```

student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ --std=c++17 task3.cpp -o t3
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t3
PID      CPU%      Memory (kB)      Name
1958     38.2894%      140732938554648%      (chrome)%
1654     16.4839%      140732938554648%      (chrome)%
1178     4.20594%      140732938554648%      (gnome-shell)%
1032     1.23095%      140732938554648%      (Xorg)%
2077     0.936944%     140732938554648%      (nautilus)%
1725     0.5233%       140732938554648%      (chrome)%
1697     0.5233%       140732938554648%      (chrome)%
2050     0.305633%     140732938554648%      (gnome-terminal-)%
1875     0.267244%     140732938554648%      (chrome)%
824      0.245586%     140732938554648%      (snapd)%

```


Day 4: Add functionality to kill processes.

Code :

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
    int pid;
    string name;
    double cpuUsage;
    long memoryUsage;
};
string readFileValue(const string &path){
    ifstream file(path);
    string value;
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}
double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}
ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
```

```

long utime=0, stime=0, starttime=0;
if(file.is_open()){
    getline(file,line);
    istringstream ss(line);
    string token;
    int count=0;

    while(ss >> token){
        count++;
        if(count==2) pinfo.name=token;
        else if(count==14) utime=stol(token);
        else if(count==15) stime=stol(token);
        else if(count==22) starttime=stol(token);
    }
}

ifstream memFile("/proc/" + to_string(pid) + "/status");
if (memFile.is_open()){
    string key, value,unit;
    while(memFile >> key >> value >> unit){
        if (key=="VmRSS"){
            pinfo.memoryUsage=stol(value);
            break;
        }
    }
}

long total_time = utime + stime;
double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
return pinfo;
}

```

```

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);
            }
        }
    }
}

```

```

        processes.push_back(getProcessInfo(pid,systemUptime));
    }
}
return processes;
}

```

```

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){
    vector <ProcessInfo> processes = getAllProcesses();
    sortProcesses(processes,true);
    cout<<"PID\tCPU%\tMemory (kB)\tName\n";
    for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryU
sage<<"%\t"<<processes[i].name<<"%\n";
    }
    int targetPid;
    cout<<"Enter PID to kill: ";
    cin>>targetPid;
    if(targetPid > 0){
        if (kill(targetPid, SIGKILL) == 0){
            cout<<"Process " << targetPid << " terminated successfully\n";
        }
    }
}

```

```

        else{
            perror("Failed to kill the process");
        }
    }
    return 0;
}

```

Output :

```

student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ --std=c++17 task4.cpp -o t4
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t4
PID      CPU%    Memory (kB)    Name
1958     37.6037%    140720940613000%    (chrome)%
1654     15.8937%    140720940613000%    (chrome)%
1178      3.88785%    140720940613000%    (gnome-shell)%
1032      1.13828%    140720940613000%    (Xorg)%
2077      0.849233%    140720940613000%    (nautilus)%
1725      0.481626%    140720940613000%    (chrome)%
1697      0.481626%    140720940613000%    (chrome)%
2050      0.277577%    140720940613000%    (gnome-terminal-)%
1875      0.245543%    140720940613000%    (chrome)%
824       0.227139%    140720940613000%    (snapd)%
Enter PID to kill: 1958
Process 1958 terminated successfully

```

Day 5: Implement real-time update feature to refresh data every few seconds.

Code :

```

#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
#include <thread>
#include <chrono>
using namespace std;
namespace fs=std::filesystem;
struct ProcessInfo{
    int pid;
    string name;
    double cpuUsage;

```

```

        long memoryUsage;
};

string readFileValue(const string &path){
    ifstream file(path);
    string value;
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}

double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
        string token;
        int count=0;

        while(ss >> token){
            count++;
            if(count==2) pinfo.name=token;
            else if(count==14) utime=stol(token);
            else if(count==15) stime=stol(token);
            else if(count==22) starttime=stol(token);
        }
    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");
    if (memFile.is_open()){
        string key, value,unit;

```

```

        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);
                break;
            }
        }
    }
    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
    return pinfo;
}

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);

processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)

```

```

        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){
    char input;
    while (true){
        system("clear");
        vector <ProcessInfo> processes = getAllProcesses();
        sortProcesses(processes,true);
        cout<<"Press 'q' to quit";
        cout<<"PID\tCPU%\tMemory (kB)\tName\n";
        for (size_t i=0;i<min(processes.size(),size_t(10));i++){

            cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryU
            sage<<"%\t"<<processes[i].name<<"%\n";
        }
        int targetPid;
        cout<<"Enter PID to kill: ";
        cin>>targetPid;
        if (targetPid == 0){
            //continue refresh
        }
        else if(targetPid > 0){
            if (kill(targetPid, SIGKILL) == 0){
                cout<<"Process "<< targetPid << " terminated
                successfully\n";
            }
            else{
                perror("Failed to kill the process");
            }
        }
    }

    cout << "\nPress 'q' to quit or Enter to refresh: ";
    cin.ignore( );
    input = getchar( );
    if (input == 'q' || input =='Q')
        break;
    this_thread::sleep_for(chrono::seconds(2));
}

```

```

    }
    return 0;
}

```

Output :

```

student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ g++ --std=c++17 task5.cpp -o t5
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$ ./t5

```

```

student@D001-38: ~/Downloads/2141019389_LSP_Capstone3
Press 'q' to quitPID    CPU%    Memory (kB)    Name
1654    14.3041%    140720539731672%    (chrome)%
1178    4.30217%    140720539731672%    (gnome-shell)%
1032    1.36354%    140720539731672%    (Xorg)%
2077    0.935957%    140720539731672%    (nautilus)%
1697    0.613033%    140720539731672%    (chrome)%
2050    0.460437%    140720539731672%    (gnome-terminal)%
1725    0.408689%    140720539731672%    (chrome)%
1875    0.20774%    140720539731672%    (chrome)%
824    0.194413%    140720539731672%    (snapd)%
1015    0%    140720539731672%    (systemd)%
Enter PID to kill: 1654
Process 1654 terminated successfully

Press 'q' to quit or Enter to refresh: q
student@D001-38:~/Downloads/2141019389_LSP_Capstone3$

```